Supplementary material for Exact Policy-Aware Regression Diagnosis for Precision-Constrained Threshold Selectors

Reference numbers correspond to the main manuscript's bibliography.

These sections retain the earlier experimental detail. They complement the main paper and the formal specification, proofs, raw records and executable audits distributed in the artifact. Timing protocols and cohorts remain separate.

S1 Fresh dependency version executions

We execute actual SciPy 1.15.3 and 1.18.1 binaries in separate pre-existing Python environments. The worker asserts its loaded SciPy version, records Python and NumPy versions, executable and module paths, extracts and hashes the interpolation implementation, and hashes its generated score arrays. It reads frozen interpolation knots, calibration values, raw tuning scores and labels; those inputs are packaged with individual hashes. The ten cold-chain calibrators and the public WDBC pilot are included without selecting cases on the new result. Training and calibrator fitting are not repeated or changed during this comparison.

Both workers call `scipy.interpolate.interp1d` on the same clipped queries and calibration arrays. Cold-chain outputs are cast to float32 as in the inherited protocol; the public pilot retains its original output dtype. We compare older 1.15.3 outputs to newer 1.18.1 outputs under the same default selector contract on [0.30,0.90]. This direction is explicitly named because the earlier artifact sometimes used the newer environment as its reference. Policy divergence is symmetric, but old/new witness membership and some threshold descriptions are directional.

All eleven worker comparisons exhibit at least one changed score, and nine exhibit some policy divergence over the agreed domain. The exact certificates are independently checked against their actual version-produced arrays. This is fresh execution evidence for sensitivity to real dependency implementations on inherited fixed data. It is stronger provenance than an engineered float32 cast, but it is not eleven independent deployments, eleven new datasets, or nine confirmed library bugs. Different numerical implementations may both satisfy their advertised numerical requirements. In the absence of an upstream issue, specified bug-fix behavior or application requirement forbidding these changes, we report a version differential and a potential application regression, not a defect in SciPy.

Default standalone reproduction re-evaluates the frozen version-worker outputs and their certificates. Re-executing both binaries requires separately installed versions, because a single Python environment cannot load the two distributions reliably as the same module name. The worker commands and environment requirements are explicit. This optional binary-reexecution requirement does not affect regeneration of the main 98-case and mutation studies from the standalone ZIP. We executed both binaries locally in this revision and preserved their fresh logs; a future reproducer is not required to trust that history to re-evaluate the included score arrays.

S2 Program conformance and the semantic information boundary

The first evaluation verifies declared old/new selector semantics. That question is useful when both implementations have faithful finite contracts, but it does not establish that an actual executable conforms to its advertised contract. We therefore add a separate program-output study. The expected specification remains unchanged for every program: whole-score-group threshold candidates, inclusive empirical precision, and maximum TP followed by maximum selected count. We never pass a mutant specification to the checker in this study.

The system under test is an executable Python module separate from the production policy comparator and independent oracle. Its build routine constructs actual Boolean candidate masks and integer counts. Its selection routine calls its own feasibility guard and uses its own priority code. Six source variants change those executable operations: strict precision, strict score thresholding, reversed secondary priority, omission of a middle group, inside-tie prefixes, and an incremented feasibility TP count. The generated source files and their SHA-256 values

are retained. Tests execute the reference and each variant on the same 30 fitted-pipeline inputs. These are source mutants of a local selector, not six pre-existing products or a naturally occurring defect collection.

The adapter instruments calls to the executable feasibility guard and records the actual k/n values it receives. It obtains these values from the running program rather than from a supplied `new_spec`. It combines those events with the unchanged reference oracle's critical ratios and the policy-domain endpoints. It then calls the executable at every resulting critical point and open-cell representative. Expected memberships come from direct threshold-mask enumeration under the reference specification. Snapshot and thirteen-point baselines call that very same executable, receive the same observed output IDs, and use the same expected specification. Their distinction is policy coverage, so this comparison removes the semantic-information asymmetry of comparing score-only tools with a configured mutant model.

This adapter has an explicit and substantial trust boundary. The corpus is audited to use alpha only as an argument to the instrumented guard; an AST check enforces that syntactic restriction for the selection routine. Candidate construction and priority do not depend on alpha. The adapter checks that guard events remain the same at all evaluated policies. Under these conditions, program outcomes cannot change between the collected guard values. The argument would fail for hidden alpha-dependent branches, stateful selection, numerical precision guards not represented by the trace, or an uninstrumented external function. A consistent trace at sampled points alone would not prove event completeness for arbitrary code. We claim coverage for this audited corpus, not a universal dynamic-analysis theorem.

Instruments do not replace the executable's Boolean result: the wrapper records guard inputs and delegates the comparison to the program's original guard. Thus the strict-precision source variant keeps its strict comparison. Candidate-family and objective mutants also run their own changed code. The unchanged oracle specification supplies the desired behavior; it does not supply the changed implementation's semantics. Each certificate records the executable source hash, frozen input hash, collected guard values, complete probe list, expected and actual membership bitsets, and an observation-ID witness when they disagree. A replay audit re-executes every stored observation and verifies the bitsets and source binding.

See Table S1 for executed local source-program conformance on 30 inputs per program; counts are separate from declared-semantics mutants.

Table S1. Executed local source-program conformance on 30 inputs per program; counts are separate from declared-semantics mutants.

Source program	Runs	Nonconforming	Snapshot	Grid	Guard-complete
drop_middle_group	30	16	2	7	16
reference	30	0	0	0	0
reverse_tie_break	30	30	22	30	30
split_score_ties	30	20	20	20	20
strict_precision	30	30	11	19	30
threshold_gt	30	10	0	10	10
tp_count_plus_one	30	30	17	25	30

Across 210 program/input executions, 136 fail reference conformance. The guard-complete adapter detects those 136, the thirteen-point baseline detects 111, and the alpha=0.50 snapshot detects 72. All 30 reference controls conform. The remaining source-mutant runs are equivalent only on their evaluated fixed input and domain; this is not global program equivalence. These totals are separate from, and must not be pooled with, the earlier 183/183 declared-semantics results. This study establishes that the method can consume outputs of real executable variants without being handed the mutant specification. It remains a local source-mutation experiment, and does not answer how well it integrates with an unrelated industrial selector.

S3 Matched input implementation costs

See Table S2 for matched-input implementation costs in milliseconds, shown as median (IQR), by pipeline and workload.

Table S2. Matched-input implementation costs in milliseconds, shown as median (IQR), by pipeline and workload.

Pipeline seed 0	Workload	Snapshot	Grid 13	Score B4	Current guard	Exact + witness
digits_svc_sigmoid	invariant	4.943 (3.331)	47.287 (26.995)	2.912 (3.362)	1.030 (0.540)	5.844 (4.451)
digits_svc_sigmoid	roundoff	2.347 (1.303)	37.954 (11.913)	1.910 (0.564)	0.661 (0.192)	7.516 (3.111)
digits_svc_sigmoid	strict_selector	1.935 (0.264)	28.913 (5.773)	2.200 (0.202)	0.538 (0.173)	6.461 (1.425)
generated_rf_quantized	invariant	0.255 (0.040)	2.949 (0.227)	0.315 (0.016)	0.530 (0.245)	1.044 (0.103)
generated_rf_quantized	roundoff	0.330 (0.074)	5.117 (2.382)	0.076 (0.017)	0.495 (0.044)	1.852 (0.062)
generated_rf_quantized	strict_selector	0.266 (0.017)	3.098 (0.497)	0.317 (0.027)	0.561 (0.068)	1.386 (0.153)
wdbc_logistic_isotonic	invariant	0.092 (0.013)	1.098 (0.101)	0.109 (0.006)	0.360 (0.051)	0.358 (0.131)
wdbc_logistic_isotonic	roundoff	0.123 (0.013)	1.346 (0.469)	0.034 (0.025)	0.374 (0.205)	0.592 (0.085)
wdbc_logistic_isotonic	strict_selector	0.108 (0.037)	1.087 (0.087)	0.103 (0.014)	0.344 (0.024)	0.424 (0.084)

Entries are median milliseconds (linear IQR), six rounds per group, 270 chronological measurements total. Every invocation rebuilds its input summaries, tracing is disabled, and method order is randomized after one warmup. The snapshot/grid use the declared SUT adapter and scan all 13 targets for the grid. Exact includes witness construction but omits JSON serialization in this comparison. Score B4/current guard flags retain their different scopes, especially for the strict-selector workload. These data describe implementation costs, not an equal-capability speed ranking. Do not pool them with the separate traced end-to-end certificate protocol.

S4 Algorithmic cost decomposition

Figure S1 separates measured computation phases and contrasts their stacked medians with the measured median total.

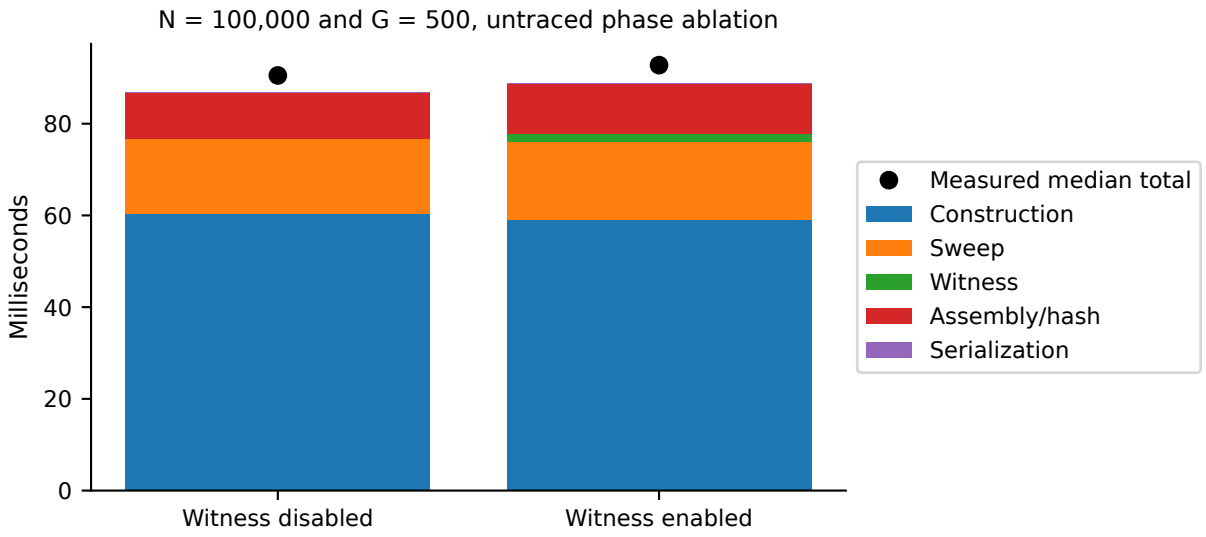


Figure S1 Phase medians and measured total. Stacked component medians need not equal the median end to end total.

End-to-end timings do not identify which part of a certificate computation consumes time. We add a within-invocation ablation with instrumented stage boundaries in the production core. Construction includes input validation, sorting, candidate profiling, exact breakpoint construction and rank summaries. The sweep phase excludes time measured inside witness scans. Witness time records only scans for distinct divergent winning-state pairs. Assembly includes PSC computation and input hashing. Serialization measures JSON encoding outside the core. The certificate returned by an instrumented call is asserted equal to the uninstrumented result on a dedicated test; profiling fields are not inserted into the certificate or used by its logic.

The ablation uses three N/G settings, witness disabled/enabled, and ten rounds per condition, for 60 raw observations. Inputs are prepared outside the timer. Each condition receives a warmup and witness-mode order is randomized within a round. Unlike the original end-to-end traced protocol, this experiment disables tracemalloc. Stage medians below therefore belong to a distinct protocol and should not replace or be pooled with the earlier traced measurements.

See Table S3 for untraced phase-ablation medians in milliseconds; component medians need not sum to the median total.

Table S3. Untraced phase-ablation medians in milliseconds; component medians need not sum to the median total.

N	Construction	Sweep excluding witness	Witness	Assembly and hash	Serialization	Total
1000	0.735	0.619	0.107	0.233	0.044	1.881
10000	5.648	4.323	0.889	1.177	0.099	13.212
100000	59.114	16.904	1.777	10.893	0.189	92.774

Values are milliseconds for witness-enabled divergent workloads. Each record also stores the measured end-to-end total and an unattributed residual. The validation checks that phase sums plus that residual equal the same invocation's total, and that witness-disabled calls record zero witness time. Medians of component stages generally need not sum to the median of totals; we do not force that identity or infer witness overhead by subtracting independent medians. The extra clock reads themselves incur overhead, and scheduler variation affects short phases.

The decomposition exposes both sorting/breakpoint work and output/witness work. Witness scans scale with the number of distinct divergent states, not just observation count. A low-cost invariant scan cannot stand in for successful witness construction. Likewise, an efficient policy sweep does not imply a constant-cost independently checkable certificate: hashing, serialization and an external mask-enumeration checker still consume resources. We keep algorithmic asymptotics, measured Python implementation costs, and the slower independent checker as separate claims.

S5 Fail closed when program assumptions are unsupported

Syntactic checks of policy-parameter placement alone cannot exclude state, indirect policy dependencies or unreviewed helpers. The adapter therefore uses explicit source admission before execution. Its registry contains hashes of only seven manually reviewed finite programs: the reference and six previously inspected source variants. The adapter additionally rejects direct alpha uses outside the designated guard, global/nonlocal state, top-level mutable state, and unreviewed decorators/defaults. Unknown source is refused even if it appears syntactically harmless. The registry is an auditable compatibility boundary, not an automatic proof that arbitrary Python is safe and not a security sandbox. Hashes do not inspect the semantics of a newly changed executable; an unreviewed source change requires a new review before it can receive a certificate.

Runtime checks require Boolean, correctly aligned membership vectors and valid integer guard counts. Transformed policy arguments, changed guard-event sets and nondeterministic repeated outputs prevent certification. Unsupported input/program behavior is a distinct outcome, not evidence of policy equivalence or a detected regression. The CLI uses exit 0 for a passing checked mode, exit 1 for observed nonconformance, and exit 2 for unsupported operation. Snapshot/grid passes are labeled only as no failure at their probes and are never marked as full-domain certified.

We test twelve unsupported source fixtures, including hidden state, alpha-dependent candidates/objectives, indirect guard access, dynamic evaluation, random priority, mutable defaults and unreviewed imports. Three additional runtime fixtures test wrong shape, non-Boolean membership and invalid guard arguments. All fifteen are refused without a certificate; all seven reviewed source programs remain admitted. These are designed compatibility/failure tests, not a measured detection rate on adversarial or arbitrary code. A safe refusal may require manual integration work even for a correct third-party program. Source allowlisting limits automation and must be treated as a deployment cost, not hidden behind a claim of general-purpose program verification.

S6 Local CI execution and frequency context

We execute actual CLI subprocesses for three pipeline representatives, the reference and strict-precision program, three probe modes and three repetitions, then six complete-mode invocations on a separate controlled N=100,000/G=500 input. The sixty timed processes are accompanied by three negative CLI checks that require exit 2 for unsupported source. All reference jobs pass and every complete-mode strict-precision job fails as expected. Partial-mode outcomes reflect only their evaluated points.

For the small pipeline representatives, the across-invocation median process time is 449.36 ms and median in-gate time is 10.29 ms. On the separate 100,000-row fixture, the corresponding medians are 4292.43 ms and 3891.59 ms. Process time includes Python startup, imports, input loading and report writing; in-gate time excludes startup and includes repeated-determinism checks, candidate masks and reference comparison. They are paired measurements from these invocations, not directly comparable to the traced compact-certificate benchmark or an estimate for a remote runner. Only three repetitions per program at this scale were measured.

This is practical local workflow evidence: CI can distinguish pass, regression and unsupported programs and attach a concrete report. It does not establish an acceptable latency budget for every team, the best testing frequency, remote GitHub execution, or a production SLO. A team could choose checks per commit or per release according to its own case count and latency budget; we make no numerical frequency recommendation from one host. Actual deployment across projects remains an external-validity limitation.

S7 Onboarding a public external selector beyond the source allowlist

We execute the installed scikit-learn `TunedThresholdClassifierCV` through its documented public API [21]. Its optimizer body is neither extracted nor edited, and it is not admitted by adding another local-program hash. A prefit score-carrier estimator exposes the frozen scores; a custom scorer returns TP*(N+1)+selected_count when the exact rational precision constraint holds and -1 otherwise. The multiplier preserves the lexicographic (TP,N) objective. Complete unique-score thresholds are passed explicitly. The adapter maps an infeasible optimum and the contract's no-positive case to abstention. These are explicit trusted integration choices, not claims about the library's default balanced-accuracy or cross-validation behavior. No predictive generalization or tuning-quality estimate is made from these fixed observations.

The workload rule is fixed before checking outcomes: seed zero from each of the three existing pipeline families, first 48 rows. WDBC, digits and random forest expose respectively 9, 48 and 13 unique thresholds. All 59 critical-point and midpoint policies conform, with 2,150 actual scorer callbacks and zero membership mismatches. The validator independently rebuilds the exact masks and executes all 59 optimizer fits again. Four configurations are explicitly refused: an integer sampled grid, a missing observed threshold, cross-validation resampling, and refitting. Full-domain coverage follows only under the complete finite candidate and exact scorer contract. It does not extend to arbitrary changes of scorer, interpolation, resampling or estimator state.

Observed onboarding effort consists of 76 physical Python lines in the executable study (including its measurement/report code), four public configuration hooks and zero upstream source edits. Source/class hashes, callbacks, decisions and execution times are retained. These are inspectable integration artifacts; programmer hours, independent team effort and maintenance costs were not measured. The scorer, score carrier and abstention mapping remain trusted glue.

The public-API study concerns a second component in scikit-learn. The eval-toolkit study uses a separate Python 3.13.18 environment to satisfy its Python >=3.13 requirement and executes the installed project as described in the main paper. Historical compatibility refusals are retained in the artifact changelog and are not counted as executed cases.

S8 Direct SMT baseline on the same equivalence contract

Beyond broad capability comparisons, we encode the external threshold-consumer task directly in Z3 [16]. Alpha is a real variable in [3/10,9/10]. A candidate is feasible under the exact linear inequality alpha*n <= k. Candidates are ordered by their fixed (TP,N) priority, and nested if-then-else terms compute the winning selected count. For this same-score task, candidates from both versions belong to one chain of score-prefix memberships, so different winning counts are equivalent to different selected memberships. That reduction does not apply to arbitrary score reorderings; we do not use it for them.

The solver asks whether any policy makes the two winning counts unequal. SAT supplies a rational policy witness; UNSAT asserts full-domain equality for the encoded task. Timeout/unknown is never interpreted as equivalence. We independently re-evaluate each SAT model with direct memberships. The complete partition checker separately emits all interval/endpoint evidence and PSC. Both therefore share a Boolean equivalence contract, while their diagnostic outputs differ. The baseline is our explicit finite SMT encoding using an established solver, not a reproduction of a published threshold-testing tool.

Across 93 variant/input tasks and three repetitions, all 279 solver verdicts agree with partition verification; there are 0 unknown outcomes and every SAT policy is independently checked. Persisted SMT-LIB queries are resolved again by the artifact validator. Median encoding time is 8.222 ms and median solving time is 0.576 ms across those queries. Counts/masks are prepared before these timers. These measurements do not establish a speed ranking against full interval construction with input validation, hashing, witnesses and JSON serialization. The direct SMT results demonstrate that full-domain Boolean comparison is also attainable with a general reasoning engine, narrowing the defensible contribution to the selector-specific contract, complete diagnostics, independently checkable output and scoped integration.

S9 Reproduction and assurance levels

The original standalone failure arose because the experiment driver read a historical 255-row grid CSV from a neighboring submission tree. The corrected driver reads a bundled `inputs/historical/` copy and asserts its checksum from the input manifest before any comparison. Default `run_all.py` works from frozen arrays within the extracted artifact; it does not refit models, reinstall dependency versions or need the old submission directory. Optional model refitting is a separate command with its source-data requirements documented. The distinction prevents the term “standalone” from silently including unbundled training assets.

Assurance has several layers. Unit/exhaustive tests validate finite implementations on small and randomized cases. The independent certificate checker reconstructs actual threshold masks on all stored application and declared-mutant certificates. The program adapter calls a separate executable and checks the unchanged reference contract under an explicit guard-coverage restriction. Fresh version workers establish what two actual dependency binaries computed. Checksummed packaging and a clean extraction run test delivery rather than mathematical correctness. None of these layers, alone or together, establishes authenticity of upstream measurements, population model quality, a universally correct verifier or an industrial real-bug detection rate.

The previous artifact is retained as a historical ZIP. Revised raw records are regenerated rather than mixed with old timings. Source snapshots bind the producer used for those records. A final manifest is generated after documents, evidence and reports have been assembled. The clean-run report names the extracted path, absence of neighboring SUBMISSION_PACKAGE, interpreter, stage exit codes and resulting logical summaries. It is a local clean reproduction, not independent human replication. A broader upstream-confirmed bug corpus and prospective deployment study remain work outside this revision; two isolated-source upstream defect case studies are now included.

S10 Detailed relation to verification and testing foundations

Parameter-region verification predates this work. Storm's official parametric-model documentation demonstrates region checking and parameter-space partitioning [6]. Junges and colleagues study parameter synthesis and coverage of parameter space for Markov models [7]. Their models, probability properties and region procedures differ from deterministic threshold membership, but they rule out presenting region partitioning itself as our invention. We do not benchmark Storm as though it were a drop-in selected-membership comparator; such a comparison would require a faithful encoding and a common property.

Regression verification also already studies equivalence of closely related program versions [8]. Our restricted finite-input selector contract makes its policy critical values explicit and directly enumerable; it does not solve general program equivalence. The program-output experiment covers a known guard-constrained corpus rather than establishing a general-purpose regression verifier. Treating the old version as a de facto specification may preserve an old bug; applications that require correctness rather than compatibility need an agreed reference contract.

The separation between a producer and a checker is likewise established. Alkassar and colleagues describe verification of certifying computations [9]. Necula and Schneck's proof-carrying-code work studies machine-checkable safety evidence and the trusted checker boundary [10]. Our certificates are concrete finite membership evidence checked by another implementation. They are not machine-checked proofs of arbitrary executable safety, and their Python checker is not a formally verified trusted kernel. An input digest prevents silent accidental mismatches but neither authenticates the original observations nor proves adapter faithfulness.

Testing ML implementations through metamorphic properties and mutation analysis is also established. The 2021 work on stability-derived relations targets linear-classification programs [11], while Xie and colleagues study classifier implementation testing and actual faults in Weka [12]. Those programs and properties differ from our fixed-label threshold selector. Their empirical designs motivate evaluating executed code and avoiding an inference from model accuracy to program correctness. Our expanded corpus addresses direct execution for a local selector, but remains weaker external-validity evidence than a curated set of independently documented, naturally occurring defects.

A focused search across these five areas expands the primary-source basis and clarifies the closest conceptual overlap. It is still not a systematic review with exhaustive database coverage and forward/backward citation closure. The appropriate novelty claim is the specialized integration of endpoint-correct finite policy equivalence, verifiable membership evidence and explicitly scoped testing adapters. A first-in-literature claim remains unsupported. The expanded paper and artifact are provided for further technical review rather than labeled ready for acceptance at a particular journal.

Supplement S8 retains the 93-task Boolean SMT study. The complete-region comparisons and policy-domain query study address richer matched outputs separately.

S11 Detailed contribution map

General parameter synthesis and AllSMT already supply machinery for exploring property regions; neither the finite partition fact nor complete-region capability is exclusive to this work. The contribution is a concrete verification interface with declared policy semantics, observation identities, endpoint ownership and input-bound diagnostic obligations, plus an independently implemented consumer and explicit integration limits. The strongest evidence is consistency under those obligations and measured engineering tradeoffs, rather than a new general solver principle.

See Table S4 for established verification foundations and the specialized contribution evaluated in this study.

Table S4. Established verification foundations and the specialized contribution evaluated in this study.

Closest foundation	What is established there	Specific work instantiated and evaluated here
Parametric verification and synthesis [6,7,20]	Reasoning about parameter-indexed properties and regions	Fixed empirical selected-ID equivalence, with exact policy/identity/numerical contracts and boundary ownership
Predicate enumeration and WMI [23-25]	Complete model-region exploration and integration	Matched selector diagnostics, exact PSC as uniform interval length and representative observation witnesses; no general enumeration novelty claimed
Certifying computations [9,17]	Producer/checker separation and evidence validation	A specified membership certificate and separately coded streaming consumer, replayed over 428 certificates with paired cost measurements

Closest foundation	What is established there	Specific work instantiated and evaluated here
Rounding preimages [26]	Arithmetic reasoning about rounding cells	A checked mapping from an actual binary64 selector interface to rational cuts, followed by allowed-policy-set significance assessment

The table identifies the implemented object and evidence; it is not a negative claim that existing tools cannot encode the same task. Our SMT baseline demonstrates that they can. The streaming data structures are established techniques, the source-replay bridges remain trusted integration code, and first-in-literature novelty is not established by this focused comparison. Two independently documented defects strengthen external evidence but do not by themselves prove a broad defect-detection advantage.

S12 Operator level mutation records

See Table S5 for operator-level declared-semantics mutation records; identity faults are reported separately.

Table S5. Operator-level declared-semantics mutation records; identity faults are reported separately.

Variant	Runs	Policy-divergent	Snapshot	Grid	ID violations
aligned_permutation_control	30	0	0	0	0
drop_middle_group	30	16	2	7	0
identity_control	30	0	0	0	0
monotone_control	30	0	0	0	0
reverse_tie_break	30	30	22	30	0
round_one_decimal	30	26	24	26	0
roundoff_tie_split	30	21	16	20	0
row_id_misalignment	30	0	0	0	30
split_score_ties	30	20	20	20	0
strict_precision	30	30	11	19	0
threshold_gt	30	10	0	10	0
tp_count_plus_one	30	30	17	25	0

S13 Repeated reuse cost protocol and limits

See results/v8/reuse_protocol.json for the predeclared selection and requests, reuse_raw.csv for 720 chronological observations, reuse_summary.csv for 144 median/IQR groups, reuse_query_plans.json for 2,028 distinct plans and reuse_outputs/ for every saved method answer. Setup includes certificate construction/JSON/full independent verification or independent mask/SMT preparation. The shared interpolation computation is outside all method timers. Request types, endpoints, batch sizes and all six inputs are retained even when a method is slower. Raw observations and deterministic query answers are separately audited. A crossing is defined only on tested batches 1/8/32/128. No developer study, production-policy inventory or universal amortization bound is claimed.

S14 Full certificate request cost protocol

We now test whether reuse pays for certificate setup, rather than treating a large query count as a productivity result. Six inputs are fixed before measurement: the upstream int8 fixture, Unix-second int32 fixture, small-int32

control, and seed zero of each pipeline family. These are existing controlled SciPy propagation inputs, not six new projects or production configurations. Two request types use distinct rational singleton policies or width-1/20 windows spread deterministically over [0.30,0.90], with varying endpoint ownership. Batches contain 1, 8, 32 or 128 requests. The scenarios test computational costs; no application policy-frequency distribution is asserted.

The common output is Boolean existence of a difference plus a rational witness. Direct masks prepare candidate memberships once, then evaluate the requested point or all relevant critical points/open cells for a window, stopping on a difference. Incremental Z3 prepares the independent selected-bitset encoding once per batch and uses push/pop for each request; unknown/timeout fails the study. The certificate route includes construction, a JSON roundtrip and independent full verification once per batch before queries. Its current API materializes intersections before projecting the Boolean answer, so that actual implementation cost remains included. No route memoizes query results; requests within a batch are distinct. All routes reuse the same executed interpolation outputs outside timing. These totals start from frozen score vectors; they exclude common SUT execution, imports, disk I/O and human investigation.

Each case/type has one warmup per method, followed by five fresh setups per batch and method with seeded randomized method order and no allocation tracing. There are 720 timing records, 144 statistical groups and 2,028 distinct case/query plans. All 30,420 method/request answers across methods and repetitions agree, and every reported rational witness is checked against independent selection. These are repeated computational observations, not independent defects or deployment trials. Raw phase costs, query outputs and median/IQR records are retained; medians are not pooled across inputs.

The table reports total median milliseconds (IQR) for batch 128, including setup. The other batch sizes and the separately measured setup/query phases remain in reuse_summary.csv.

See Table S6 for full-certificate batch-128 request costs in milliseconds, shown as median (IQR), including setup.

Table S6. Full-certificate batch-128 request costs in milliseconds, shown as median (IQR), including setup.

Workload	Request	Checked certificate	Direct masks	Incremental Z3
Upstream int8	Point	9.49 (1.84)	2.65 (3.24)	125.92 (9.24)
Upstream int8	Window	26.04 (1.75)	18.96 (1.72)	198.72 (26.04)
Unix int32	Point	11.47 (5.35)	3.61 (0.35)	149.92 (41.33)
Unix int32	Window	20.02 (5.50)	15.70 (1.66)	129.55 (29.31)
Small int32 control	Point	9.26 (1.04)	3.63 (0.87)	43.07 (8.67)
Small int32 control	Window	8.12 (1.80)	15.74 (0.33)	38.59 (3.91)
WDBC seed 0	Point	17.45 (2.96)	5.91 (1.27)	206.53 (21.52)
WDBC seed 0	Window	100.67 (34.56)	22.59 (6.40)	302.73 (47.63)
Digits seed 0	Point	72.81 (6.03)	21.45 (1.89)	366.95 (61.65)
Digits seed 0	Window	294.57 (59.74)	42.72 (19.33)	424.42 (138.27)
RF seed 0	Point	27.40 (10.63)	10.77 (0.15)	334.45 (35.30)
RF seed 0	Window	52.09 (4.23)	21.08 (1.96)	290.38 (52.00)

At batch 128 the checked certificate is faster than direct masks on 0/6 singleton and 1/6 window inputs; it is faster than this incremental Z3 encoding on 6/6 and 6/6 respectively. Observed sustained crossings against direct masks are Small int32 control / continuous_window at batch 32. A sustained crossing means the first tested batch with a lower certificate total median at that and every larger tested batch; it is a discrete observed comparison, not an inferred exact threshold, significance test or extrapolation beyond 128 requests. Five repetitions on one host do not establish a population advantage.

The mostly negative comparison against direct masks constrains the practical claim. Reuse alone does not justify building a certificate for a small finite decision set or guarantee an advantage even for continuous windows. A separately checkable, persistable diagnostic object may be required for exact localization or handoff, but this experiment does not monetize that requirement or measure a developer's debugging time. It measures a computational tradeoff against specified implementations, including a reused solver, and leaves industrial policies, human triage benefit and superiority over an independently optimized parametric tool unresolved.

S15 Boolean API study and comparison panels

The exact protocol is results/v9/boolean_protocol.json. Both certificate routes pay construction, JSON roundtrip and the same independent full verification; the indexed route also pays copying/freezing/index construction. This is the assurance-matched API panel. Direct masks and incremental Z3 answer the same Boolean question without emitting/checking a complete diagnostic object, so four-route totals are a separate request-cost panel with unequal evidence obligations. The original six inputs and 2,028 plans are reused: 960 timing records, 192 groups and 40,560 repeated method answers are not new workloads or defects. Full per-phase medians/IQRs and raw observations are retained. A descriptive speed ratio is not a statistical or deployment guarantee. BOOLEAN_QUERY_AND_ASSURANCE.md gives component-merging/endpoint reasoning, asymptotic scope and refusal behavior.

S16 Scope and overlap clarification

The scientific implementations and the 33-stage protocol are bound in provenance/v9_science_source_hashes.json. Reference [29] is the authors' confirmed unpublished related manuscript, separate from the 28 external sources. The artifact retains the overlap map, publication-status provenance and detached execution records. Revision history is retained in docs/ARTIFACT_CHANGELOG.md alongside the current manuscript sources.

S17 Cross platform source snapshot integrity

Generated mutant and refusal-fixture Python sources use explicit UTF-8 bytes and LF newlines, followed by byte-hash verification. Execution-source copies retain actual source bytes. The default 33-stage chain is self-contained; revision-named files are cumulative study components, not dependencies on old release ZIPs. Historical source-binding counts and repairs are documented in the artifact changelog. Native Linux execution has not been claimed.

S18 Core proof obligations and numerical interface semantics

S18.1 Contract and complete finite partition

Fix $N > 0$ aligned observations with unique IDs, binary labels and finite old/new scores. Each version has its own finite family of nonempty threshold candidates, indexed by s as in Main Section 2.1. Candidate j has actual positive count k and size n; equal scores enter together. Inclusive feasibility compares the rational policy with k/n. Priority is a fixed total order independent of the policy; for the reference contract it is lexicographic in actual positive count and selected size. No feasible candidate or an all-negative label vector gives the empty selected set. Alternative declared contracts may use different fixed feasibility coefficients; these must be enumerated separately and kept distinct from the actual-positive priority. Policies lie in $A = [L,H]$, with rational $0 <= L < H <= 1$. These assumptions concern fixed empirical observations, not population precision or arbitrary executable behavior.

Proposition S1 — finite partition.

Let B be {L,H} together with every exact candidate feasibility ratio from both versions strictly inside A. Partition A into the singleton at each element of sorted B and the open intervals between consecutive elements. On any open atom, alpha cannot cross a feasibility ratio, so every candidate feasibility predicate is constant. Fixed priority and abstention then make the winning membership constant. Thus equality at every singleton and one point of every nonempty open atom is necessary and sufficient for equality on A. Evaluating singletons separately preserves inclusive endpoint ownership; strict feasibility changes their outcome but not the required critical set. A midpoint argument is sound only after deriving the complete B independently.

S18.2 Prefix membership equality and producer cost

Lemma S2 — equal finite prefixes.

Let the old and new winners be prefixes of their respective score orders, with sizes n and m and zero-based new ranks r_new(i). If n != m, the sets differ. If n = m = 0, both are empty. Otherwise equality holds exactly when every old selected ID has new rank below n. Necessity follows from equality. For sufficiency this rank condition gives containment in the new prefix, and equal finite cardinality gives equality. Prefix maxima of new ranks in old order answer this condition in constant time after linear preprocessing. A changed-membership ID is a directly checkable witness; a single witness does not enumerate every changed ID.

Proposition S3 — sweep and complexity.

As alpha increases, a candidate that becomes infeasible never becomes feasible again. Start with all candidates in a fixed-priority heap and discard infeasible top candidates until the top is feasible or the heap is empty. Each candidate is inserted once and removed at most once; no monotonicity of empirical precision along the score order is needed. Sorting scores, forming the critical union and sweeping K atoms therefore cost O(N log N + G log G + K) under unit-cost rational arithmetic, where G is the total candidate count and K is the critical-value count up to a constant-factor number of atoms. Retained ranks, candidates and cells require O(N + G + K) auxiliary storage. Scanning N rows for each of D distinct divergent winner pairs adds O(ND) witness/count work. Fraction comparison has operand-dependent bit cost; hashing and serialization are additional measured work. A higher-priority feasible candidate cannot be hidden below an infeasible heap top after all infeasible tops are discarded.

S18.3 Streaming independent consumption

Lemma S4 — suffix winner.

The consumer independently enumerates each actual threshold mask, counts its positives and size, records its exact feasibility ratio and fixed priority, then releases the mask. Sort descriptors by nondecreasing feasibility ratio p. At alpha, inclusive feasibility is precisely the suffix beginning at lower_bound(p, alpha), while strict feasibility begins at upper_bound(p, alpha). Reverse induction computes the maximum priority of each suffix: its winner is the better of the current descriptor and the already computed next-suffix winner. The query therefore returns the same winner as exhaustive enumeration. Within a nested candidate family, equal selected size identifies the same prefix, so a priority tie does not hide another membership.

Proposition S5 — consumer obligations and cost.

First validate the input digest, declared contracts, ordered coverage and unique endpoint ownership. Independently derive all critical values and probe them and one midpoint per open atom; additionally check certificate boundaries and their open-cell representatives. Proposition S1 and Lemma S4 supply the correct winner on every atom. Reconstruct current winner memberships directly from original scores and descriptors; compare actual bitsets, not just counts. Check each cell verdict, witness ID and membership, changed-row count and the independently accumulated divergent length and Boolean equivalence. Acceptance satisfies the finite-domain obligations conditional on these algorithms and their implementations; the claim is not a proof-assistant verification. Only current masks and descriptors are retained, giving O(N + G + K + C) auxiliary descriptors plus operand storage for C certificate cells. Threshold enumeration and winner reconstruction can still require O(NG) work; ratio sorting and bisections add O(G log G + (K+C) log G), and probe sorting adds O(C log C + K log K). Split-tie ordering can add O(N log N). Quadratic work remains possible when G = N.

S18.4 Exact binary64 guard cuts

Lemma S6 — nearest-even guard preimage.

Let p be an actual finite nonnegative binary64 empirical precision returned by the public selector, q its next larger binary64 value and c the exact rational midpoint of p and q. Convert a rational policy alpha with CPython nearest-even float(alpha). Monotonic rounding maps every alpha below c to a value at most p and every alpha above c to a value at least q. At c the even significand wins. Consequently the inclusive public guard p >= float(alpha) holds below c, and at c exactly when p is the even neighbor. A public guard therefore has an exact rational cut with explicit closure. Union these cuts with the exact-reference k/n cuts and domain endpoints; Proposition S1 applies again because the candidate source, scores, labels and priority are policy independent. Actual public-selector calls

at every resulting breakpoint and midpoint check this guard mapping on the recorded runtime. This argument excludes alternative rounding modes, hidden state and additional policy-dependent branches. A rational/binary64 interface differential alone is not a confirmed upstream defect.

S18.5 PSC and endpoint-only divergence

Corollary S7 — length does not establish equivalence.

Disjoint open atoms have additive exact rational lengths, while singletons have zero Lebesgue measure. PSC is the sum of divergent open-atom lengths divided by H-L. Full equivalence implies PSC = 0, but zero PSC does not imply equivalence. For labels [1,0], scores [1,0] and A = [1/2,1], inclusive and strict feasibility select different memberships at 1/2 and at 1, but agree on the intervening open interval. Their PSC is zero. Certificates must therefore retain the Boolean verdict and endpoint ownership independently of the length statistic.

S18.6 Trust boundary

The producer, consumer and executable adapters share a mathematical specification and numerical dependencies; implementation separation is not independent human replication. A source adapter must establish the admitted candidate and guard contract; agreement on sampled traces cannot prove arbitrary hidden program semantics. SHA-256 binds supplied bytes, not dataset authenticity or authorship. The original arrays and agreed selector contracts remain necessary to check memberships. The artifact proof documents provide implementation detail beyond these self-contained obligations.

S19 Documented upstream replay details

These protocols and records support Main Sections 2.4–2.5 and 3.2. Historical-source replay is distinct from historical binary installation.

To move beyond a locally designed source-mutant corpus, we add scikit-learn issue #23213 [14] and its merged fix #23214 [15]. The confirmed defect concerns truncation of the precision-recall curve after first reaching full recall. A curve consumer can consequently miss lower thresholds that retain all positives while adding further observations. That matters to this paper's explicit secondary objective: among candidates with maximum TP, prefer the largest feasible selected set. The omission is not automatically a defect in every downstream application; we evaluate its propagation under this particular, unchanged consumer contract.

We freeze the full upstream metrics source file at scikit-learn 1.0.2 commit `7e1e6d09bcc2eaeba98f7e737aac2ac782f0e5f1` and at the PR merge commit `32c53bc69cc0d1f142700dd6c02281a000b855b9`. SHA-256 values, upstream URLs, and BSD license files accompany them. We compile and execute the original `precision_recall_curve` function nodes without rewriting their bodies. The installed scikit-learn 1.9.1 supplies its public counting routine through a compatibility bridge; every returned precision/recall value is checked against direct membership counts. This is isolated historical-source replay, not installation of an old release binary. We also execute the installed current curve implementation and check its returned thresholds against the fixed-source result. A 1.1.0 snapshot inspected during provenance checks still contained the truncation path, so we do not incorrectly label that tag as the fix.

The frozen inputs comprise the upstream report's synthetic-data/logistic protocol fitted using the current environment, plus the same thirty pipeline workloads used earlier. The report protocol fits and evaluates its classifier on the same rows to expose a metrics defect; no generalization claim follows. One bug exercised on these inputs remains one independently documented defect, not thirty-one independent software projects or defects. Historical fitted coefficients and a historical binary environment were not reproduced.

All methods receive the same actual upstream threshold outputs and the same exact TP/N feasibility and selected-membership objective. The common consumer adapter compares the complete-group reference family to the returned threshold family. It excludes the plotting sentinel that has no corresponding threshold. It derives exact integer counts from the associated masks rather than mistaking floating-point precision rounding for the upstream truncation bug. This curve-consumer adapter is separate from the local alpha-guard program adapter; it does not imply that arbitrary upstream Python passed our local source-admission policy.

See Table S7 for historical scikit-learn precision-recall defect and downstream policy effects.

Table S7. Historical scikit-learn precision-recall defect and downstream policy effects.

Evidence	Result
External projects and upstream-confirmed defects	1 project, 1 defect
Input workloads and variant/input executions	31 workloads, 93 executions
Affected-source workloads with policy divergence	15
Snapshot / grid / full-domain detections on those workloads	14 / 15 / 15
Fixed-source and installed-current policy false alarms	0

SciPy issue #6318 [27] reports incorrect nearest-neighbor interpolation for integer axes; merged PR #6351 [28] fixes overflow in midpoint construction by dividing before addition. We freeze the exact pre-merge parent `3565a801587a699d2f320a6406d0f38f86c0c4b0` and merge `7a46deb6b272fb8cdf4348cb3db45ca0ef3b2636`, with source, upstream tests, SHA-256 values and BSD license. The original regression test uses int8 knots [0,50,127] and requires interpolation at those knots to reproduce their values. Executing its unchanged test body fails on affected source and passes on fixed source and installed SciPy 1.18.1.

We compile the original interp1d class and its two fill-value helpers without rewriting method bodies. A compatibility namespace maps the removed NumPy float_ alias to float64 and supplies the modern _Interpolator1D base helper. Only the nearest branch on the stated finite one-dimensional inputs is evaluated. AST/source digests and the helper hash are recorded. This is isolated historical-source replay with a transparent bridge, not installation of an old SciPy binary, the reporter's complete original time series, or general equivalence of the historical and modern libraries. The overflow-causing integer addition remains unchanged; the matched fixed/current controls reproduce knot scores exactly.

The six prespecified axis fixtures include the upstream int8 axis, derived int16, Unix-second int32 and aligned large int64 axes, plus small-int32 and float64 controls. We additionally attach synthetic int32 Unix-second axes to the first 64 rows of all thirty existing pipeline score vectors and query at the observed knots. The scores/labels remain frozen; these row-index time carriers are constructed propagation fixtures, not independently collected time-series applications. Correct interpolation at observed knots is an identity requirement. The unchanged downstream consumer then compares selected IDs under the same rational precision objective.

See Table S8 for historical SciPy interpolation defect and downstream policy effects.

Table S8. Historical SciPy interpolation defect and downstream policy effects.

Evidence	Result
Workloads / variant-workload executions	36 / 108
Affected-source score changes / policy-divergent workloads	34 / 31
Snapshot / grid / full-domain detections on divergent workloads	26 / 31 / 31
Fixed-source and current policy false alarms	0

S20 Detailed production and larger candidate costs

Main Section 2.5 defines the timing boundaries and Main Section 3.3 reports the central comparisons. The tables below retain all detailed conditions moved from the earlier Main reading copy.

See Table S9 for traced certificate-production cost by scale, workload and witness mode; medians and IQRs remain within each condition.

Table S9. Traced certificate-production cost by scale, workload and witness mode; medians and IQRs remain within each condition.

N	G old (nominal bins)	Workload	Witness enabled	Median ms	IQR ms	Peak aux MiB	Witness cells
1000	20	divergent	False	9.55	1.53	0.10	0.0
1000	20	divergent	True	8.26	1.55	0.10	4.0
1000	20	invariant	False	4.81	1.05	0.09	0.0
1000	20	invariant	True	4.99	0.84	0.09	0.0
10000	100	divergent	False	62.46	14.70	1.00	0.0
10000	100	divergent	True	63.02	22.87	1.00	29.0
10000	100	invariant	False	35.65	2.15	0.91	0.0
10000	100	invariant	True	34.97	1.71	0.91	0.0
100000	500	divergent	False	520.59	120.04	9.22	0.0
100000	500	divergent	True	528.06	40.73	9.24	111.0
100000	500	invariant	False	368.19	103.85	8.80	0.0
100000	500	invariant	True	341.13	60.75	8.80	0.0

All rows use ten measured repetitions after warmup on one Windows host, Python 3.12.10/NumPy 2.3.5, with randomized mode order, tracemalloc enabled and JSON serialization included. Peak is auxiliary MiB with preallocated inputs excluded, not process RSS. The N=100,000 divergent witness-enabled median is 528.06 ms (IQR 40.73), emitting 111 witness-bearing state cells; the corresponding witness-disabled median is 520.59 ms (IQR 120.04). These runs activate witness construction, unlike the earlier one-policy invariant guard benchmark. No significance, independence, universal speed advantage, or direct cross-protocol latency ratio is asserted. Raw chronological records and exact seeds/generation code are included.

The eight external affected/fixed tasks remain a separate check and still agree. We add a prespecified factorial design: N in {1,000,10,000,100,000}, G in {10,30,90}, and no, one-middle, or alternating candidate omission. Scores form G tied groups with gradually decreasing group-positive proportions; fixed-seed permutations remove dependence on row order. These designed families yield 0–39 divergent components. They are controlled tasks, not new upstream defects or independent deployments. Every one of the 27 tasks is retained, including empty-divergence controls.

Five repetitions per method/task, with randomized method order inside each repetition, produce 270 timing records and 54 saved certificates. Both methods receive the same candidate family, regenerate counts/masks during each timed preparation, emit the same diagnostic JSON and invoke the same independent region checker. All complete outputs agree and every saved certificate is replayed. Preparation, producer plus serialization, checker, total checked cost, candidate counts, exact critical-point count, divergent components, probes, bytes and solver calls are retained separately. Thus input-size effects are not inferred from a preparation-excluded timer alone.

The previous G<=90 factorial is extended with six additional prespecified tasks at N=10,000, G in {250,500} and the same none/one/alternating omission patterns. Both algorithms still produce identical complete diagnostics. Three repetitions per method, randomized within each replicate, yield 36 raw timing records and twelve independently replayed certificates. No task is dropped for being unfavorable. Residual UNSAT is required for SMT completion; unknown or timeout would prevent certification.

See Table S10 for complete diagnostic comparison for larger candidate counts at N = 10,000; three repetitions per method.

Table S10. Complete diagnostic comparison for larger candidate counts at N = 10,000; three repetitions per method.

G / omission	Critical points	Components	Partition total ms	SMT total ms
250 / none	215	0	986.40	797.28
250 / one	215	1	735.13	918.14
250 / alternating	215	107	973.93	3649.18
500 / none	430	0	3618.14	2268.31
500 / one	430	1	3993.99	4715.14
500 / alternating	430	215	5291.80	21336.96

Totals include fresh candidate preparation, complete region/witness production plus JSON and the same region checker. The new cases reach 430 critical points and 215 divergent components. Three repetitions are descriptive and are not pooled with the older five-repetition protocol. The prepared-membership region checker is distinct from the streaming consumer for the core certificate schema. These results extend the evaluated candidate range, but do not establish asymptotic superiority, a deployment SLO, or a win over an independently optimized AllSMT implementation. The baseline remains our direct Z3 encoding; modern enumeration methods [25] retain capabilities and optimizations this study does not benchmark.

S21 Numerical interface distributions and policy lattices

These records complement Main Sections 2.2 and 3.4. Rational decimal policies and exact embeddings of representable binary64 policies have different meanings.

We build a wheel from the complete, unmodified eval-toolkit 1.12.0 source at commit `a6400e74c4534d6d79f17dee11ad903b6d664980`, retaining hashes, its MIT license and package metadata. Python 3.13.18 satisfies its >=3.13 requirement. The installed TargetPrecisionSelector is called through its actual public API; no method body is extracted, patched or replaced with a custom scorer. All eighteen packaged upstream threshold tests pass. The package and source are distributed for offline installation. The external case studies now execute code from scikit-learn, eval-toolkit and SciPy; eval-toolkit depends on scikit-learn's precision-recall routine, so this is not independent numerical infrastructure or independent institutional deployment.

The workload rule is fixed: first 64 rows of all thirty existing pipeline inputs, with no positive-case filtering. The wrapper passes float(alpha) to the public precision parameter and maps only the documented no-feasible RuntimeError to abstention. Its exact-reference objective is still maximum (TP,N); in the same nested family, this equals the project's smallest-feasible-threshold objective. Input validation, source identity and candidate counts are checked. Actual floating precision values are compared against direct integer membership counts.

It would be unsound to assume a binary64 guard changes exactly at the rational ratio k/n. For an actual binary64 precision p, let q=nextafter(p,+infinity) and c=(exact(p)+exact(q))/2. The public predicate p >= float(alpha) changes at c, including equality exactly when p has the even significand. The proof follows monotone nearest/even rounding and is a selector-specific application of established rounding-preimage reasoning [26]. We explicitly check midpoint ownership and both neighboring cells on the runtime. The complete critical set is the union of rational-reference cuts, these rounding-cell cuts and domain endpoints. A model of these guards is checked against every actual public-API output, including all endpoints; unsupported numerical behavior must prevent certification.

Each input-bound certificate preserves exact rounding-cut fractions, ownership, actual/reference bitsets, row witnesses, complete divergent regions and PSC. The validator calls the installed selector again at all 884 policies and checks every stored field. We additionally probe actual representable binary64 precision policies: 196 such probes expose differences on 19 workloads. Grid/snapshot targets are exact rational fractions, whereas this additional group embeds each binary64 value by its exact integer ratio; it does not reinterpret a decimal grid target as its binary approximation. Thus the continuous-domain finding is not automatically equated to a difference at a policy the float API can represent. With labels [1,1,0,0,0] ordered by decreasing distinct score, the representable float 0.4 exceeds exact 2/5: at the exact rational embedding of that float, the float selector admits all five while the

rational reference admits four. At exact decimal 2/5 both admit five. This is a designed explanation of distinct policy contracts, not a newly documented upstream bug. Tiny interval length is not a deployment probability or impact estimate. Adoption should decide whether binary64 semantics are acceptable, rather than treating every differential as a release-blocking defect.

The twenty-seven numerical-contract differentials have extremely small policy measure. We compute all quantiles using exact fractions and linear interpolation before converting only display values to scientific notation. Three further workloads have zero PSC; Boolean equivalence and endpoint evidence remain separate properties. The distribution below is conditional on the 27 positive-PSC workloads, not all thirty observations and not a probability distribution of deployed policies.

See Table S11 for exact positive-PSC distribution for numerical-interface differentials, conditional on 27 positive workloads.

Table S11. Exact positive-PSC distribution for numerical-interface differentials, conditional on 27 positive workloads.

Positive-PSC statistic	Value
min	1.541976e-17
q25	1.659852e-16
median	3.747190e-16
q75	6.795052e-16
max	1.660140e-15

At individual verified policies, membership can still change: across these fixtures the maximum changed-ID count is 41 of 64. The largest precision shortfall observed at stored probes is 1.048544e-16; separately, the exact supremum across all covered cells is 1.048544e-16. That supremum includes limits at open cell endpoints and need not be attained. These quantities do not turn a tiny arithmetic discrepancy into demonstrated harm. The observed labels provide no deployment-frequency, utility, monetary loss or human-outcome data.

We intersect the exact divergent components with four explicitly declared policy lattices on [0.30,0.90]. Policies are rational decimal targets, not exact embeddings of their binary64 approximations. Intersection is computed algebraically from integer bounds and endpoint ownership; it does not claim to make millions of public-API calls. Earlier complete partition validation supplies coverage of the intervals, and the validator cross-checks closed/open lattice boundaries.

See Table S12 for exact intersections with declared decimal policy lattices; these are study scenarios rather than deployment frequencies.

Table S12. Exact intersections with declared decimal policy lattices; these are study scenarios rather than deployment frequencies.

Decimal places	Allowed policies per workload	Workloads with an affected policy	Affected policy/workload pairs
2	61	0 / 30	0
3	601	0 / 30	0
4	6,001	0 / 30	0
6	600,001	0 / 30	0

For these measured workloads, applications restricted to any one of those decimal lattices have no affected allowed policy under the declared reference/interface mapping. That is equality on a specified finite policy set, not continuous-domain equivalence or a proof that arbitrary workloads are safe. Conversely, zero interval length can still include an allowed singleton; a dedicated boundary test prevents confusing PSC=0 with finite-set

equality. The nineteen representable-binary64 examples use a different exact policy embedding, as explained above, and are not silently substituted for decimal targets.

The practical role of the tiny-region certificates is therefore narrow: auditing a strict numerical contract, documenting a numerical-interface migration, or deciding whether an explicitly permitted policy set intersects a difference. They do not demonstrate an extra bug or justify blocking every routine CI job. An application may accept the binary64 contract or define a restricted decimal policy set; such a choice must be declared before interpreting a report. We preserve the exact diagnostic result rather than erasing small regions with a tolerance and calling the versions universally equivalent. The separate SciPy defect study shows broad policy effects, but its constructed propagation cases likewise do not supply operational impact or deployment rates.

If an application already declares a small finite allowed set, exhaustive checking of every member is itself complete for that set. Continuous-domain certification is not required for completeness on that set; Main Section 3.4 evaluates its cost tradeoff separately. Its additional value is the reusable exact region description and contract-localized explanation; its actual frequency and human utility remain unmeasured.

The same exact permitted-set assessment is applied separately to the SciPy defect fixtures: at two decimal places, 31 of 36 workloads have affected targets, comprising 1,611 of 2,196 allowed policy/workload pairs. Per-case counts for all four resolutions are retained in scipy_allowed_policy_counts.csv. These are constructed-fixture coverage counts, not observed deployment frequencies or independent defect counts.

S22 Detailed comparison with verification and testing foundations

The table distinguishes conceptual foundations from executed comparisons and complements Main Sections 1.2 and 4.2. Complete-region and Boolean request-cost protocols have different output and assurance obligations.

See Table S13 for executed method comparisons, conceptual foundations and the limits of the evidence.

Table S13. Executed method comparisons, conceptual foundations and the limits of the evidence.

Method considered	Matched question here	Evidence actually compared	Remaining distinction
Snapshot and thirteen-point probes	Membership compatibility at chosen policies	Executed on identical SUT outputs and contract	Cannot certify unprobed policies
Direct Z3 encoding	Existence of any divergent policy on finite nested candidates	93 actual equivalence tasks, rational counterexample rechecks	One counterexample/Boolean rather than all intervals and PSC
SMT predicate-cell enumeration [16,23]	Complete divergent regions, exact PSC and representative witnesses	Eight external, 27 factorial and six larger-G tasks; complete outputs and checker agree	Same-score nested candidate encoding and finite feasibility predicates
Our partition and checker	Same Boolean task plus full divergence localization	Exact endpoints, intervals and independently replayed memberships	Specialized finite contract and explicit adapter assumptions
Certifying computations [9,17]	Trust in a particular computation and checker	Primary-source conceptual comparison	General foundation, not an empirical competing selector tool
PROPhESY/Storm [6,7,20]	Parameter-region property verification	Primary-source property/input-model comparison	No faithful probabilistic encoding benchmark claimed
Parametric monitoring [19]	Monitoring properties across parameter instances	Primary-source semantic comparison	Object/trace parameters differ from empirical precision breakpoints
Robust classification [18]	Classifier choice as operating conditions vary	Primary-source objective comparison	Does not make the finite-breakpoint idea novel